

Vision-Only Reactive Corridor Navigation for a Quadruped using a Frozen Velocity Policy and a Hard Passage Gate

Mohammadreza AhmadiTeshnizi, Patrik Perčinić (Team Ava)

Robotics 2026, LIACS, Leiden University

Video: github.com/teshnizi2/go2-ball-follower/blob/main/demo/go2_submission.mp4

Code: github.com/teshnizi2/go2-ball-follower

Abstract. A simulated Unitree Go2 quadruped follows a moving red ball down a long, cluttered corridor using only its head camera. It has no GPS, no lidar, no depth sensor, and no map of the corridor. We did not train the robot to walk. We take a pre-trained reinforcement-learning velocity policy, leave its weights untouched, and build three things around it: a colour-based ball tracker, a reactive planner, and the obstacle course itself. The key idea is small. The walking policy accepts a sideways velocity command, but a controller that only walks toward a target never uses it, so the robot ends up behaving like a car that can only change lanes by turning. We switch that sideways channel back on, so the robot can slide into a gap without turning to face it. We add one more rule: the robot may not enter an obstacle row until its body is already lined up with the gap. With these two changes, and a corridor that always leaves at least one passable lane, the robot runs for ten minutes (600 s) on nine random layouts and never touches an obstacle or a wall, at about 0.6 m/s. To keep the task honest we let the ball travel through the obstacles instead of around them, so the robot has to do all of the avoiding by itself.

1. INTRODUCTION

Most robots that move through cluttered spaces rely on rich sensing (lidar, depth cameras, or motion capture) and a mapping and planning stack on top. For this project we set ourselves a tighter rule and asked a simple question. How far can a four-legged robot get through a dense obstacle corridor if all it has is one RGB camera on its head and a walking controller it did not train?

The robot chases a moving red ball, which stands in for a goal to walk toward. The corridor is five metres wide and packed with rows of obstacles. The obstacles get wider and the gaps get tighter the further the robot travels. We do not train the legs ourselves. The walking comes from a frozen, pre-trained reinforcement learning (RL) policy from the RSL-RL framework [1]. The ball tracking is plain colour vision. Everything in between, the planner, the safety rules, the corridor generator, and the on-screen dashboard, is ours.

What is new here. The policy from Rudin et al. [1] is omnidirectional: it takes a command made of forward speed, sideways speed, and turn rate (v_x , v_y , v_ω). A controller that just "walks toward the ball" only ever sets the forward speed and the turn rate. That leaves the robot non-holonomic, like a car: to move sideways it first has to turn, walk, and turn back. Our point is that turning the unused sideways channel back on, together with a rule that refuses to enter a gap before the body is aligned, is what lets an untrained walker thread a tight corridor reliably. We did not find this particular combination, reusing a spare command channel of a frozen policy purely for reactive, single-camera obstacle avoidance, in the work we read.

2. RELATED WORK

Learned legged locomotion. Reinforcement learning has produced strong quadruped controllers in simulation and on real hardware [2], including blind walking over rough terrain [3] and training schemes that learn velocity-command policies in minutes [1]. We treat one of these policies as a black box that turns velocity commands into joint motion, and we never change its weights.

Reactive navigation. We do not build a map. Instead the planner looks at the next row of obstacles, picks a gap, and steers toward it with a pure-pursuit law [4]. This is closer to follow-the-gap and potential-field methods than to global planning, which suits a robot that has little sensing and has to react fast.

Tracking the ball. The ball is found with an HSV colour mask and followed with CamShift [5], a light histogram tracker. We picked it over a neural detector to keep the pipeline simple and easy to inspect. The simulator is MuJoCo [6] and the Go2 model comes from the MuJoCo Menagerie.

3. SYSTEM DESIGN

The robot is three layers wrapped around the frozen walking policy (Fig. 1).

- **Perception** (`tracker.py`) finds the red ball with an HSV mask and follows it with CamShift. It resets and re-detects when its confidence drops.
- **Reactive planning** (`controller.py`, `main.py`) chooses a gap through the next row, sticks with that choice, and turns it into a body-frame velocity command, including the sideways slide. A pure-pursuit law follows the resulting path. A hard gate and a push-away term form the safety net.
- **Walking** (`low_level.py`) is the frozen policy. It reads a 45-number observation plus the velocity command and outputs 12 joint targets at 50 Hz. A PD controller tracks those targets at 500 Hz.

A separate part of the code builds the corridor. It lays out the obstacle rows, ramps the difficulty with distance, and moves rows the robot has passed back to the front so the course never ends.

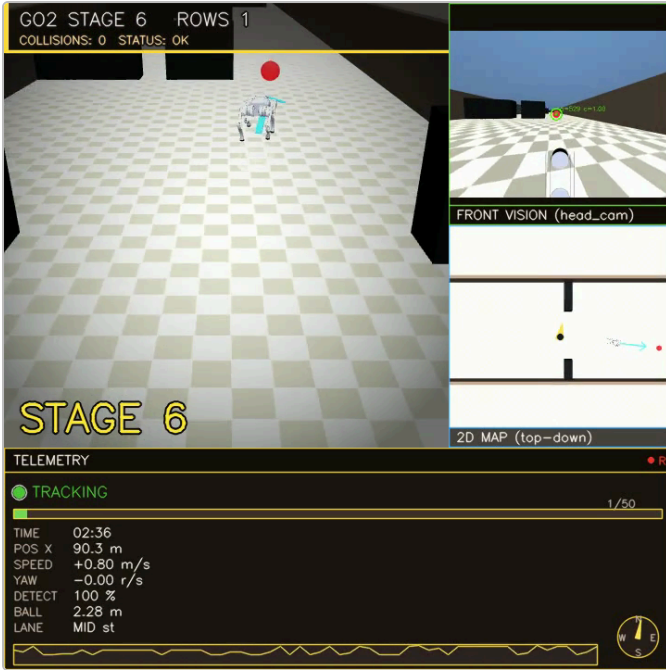


Fig. 1. The 720x720 dashboard. Large panel on the left is the chase view, with the planned path drawn as an arrow on the floor and a halo around the obstacle being avoided. Top right is the robot's own head camera with the tracked ball. Below it is a top-down map. The bottom strip shows speed, range, detection confidence, a status light, and the current stage.

4. IMPLEMENTATION

4.1 Finding the ball

Red wraps around the hue circle, so we use two HSV ranges and clean the mask up with morphology. The largest sensible blob gives a starting box and a hue histogram, and CamShift tracks that box from frame to frame. Short occlusions are fine because the tracker coasts; a long loss forces a fresh detection. In the simulator we also know exactly where the ball is, so the controller can keep chasing even during the moments the ball is hidden behind an obstacle.

4.2 The planner and the gate

For the next row, the planner grows each obstacle by a safety margin (roughly the robot's half-width plus a little slack), removes those blocked strips from the corridor, and keeps the widest gap it can actually reach. It aims at the middle of that gap, never the edge, and it stays committed to the gap until the robot is fully past the row. Committing matters: without it, the ball can pull the robot back into an obstacle it has just cleared.

The part that really makes it work is the gate. While the robot is not yet inside the chosen gap, it is not allowed to drive past the front of the row. Its forward speed drops to zero at that line while a strong sideways command slides it into place. Only once it is lined up does it move through. Because the gate forces the robot to line up before it enters, the robot stays safe no matter where the ball leads it.

4.3 Turning the sideways channel back on

We convert the high-level command into the robot's own frame and feed the sideways part into the policy's v_y input. Our first version left that input at zero. Switching it on lets the robot step sideways into a gap near the wall instead of doing the slow turn-walk-turn dance. On top of that, a push-away term nudges the robot sideways whenever its body gets close to an obstacle edge,

as a backup. We also cap the forward speed during hard turns, because the frozen policy gets shaky in that corner of its command range. If the robot does trip, it stands back up where it fell instead of restarting the run.

4.4 The corridor and the difficulty curve

Every row is built so that at least one lane is wide enough for the robot to fit through. This matters because two very wide obstacles can otherwise split the corridor into slivers that no robot could pass, and the planner would just get stuck. Difficulty grows with distance, not time: obstacles get wider, the guaranteed lane gets narrower down to a floor, and the colour darkens. This happens over 17 stages that climb and then hold. Rows the robot passes are recycled to the front, so the corridor is effectively endless. The ball follows its own gentle sine path and ignores the obstacles, walking straight through them, so the robot cannot just trail a pre-cleared route. It has to avoid on its own.

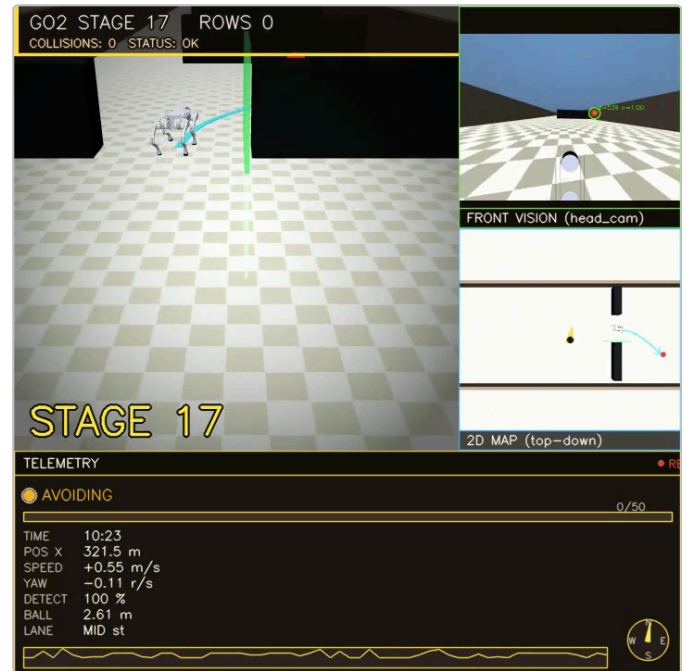


Fig. 2. Late in a run (stage 17, about 340 m in). Wide black obstacles fill the middle of the corridor and the robot threads the lane that is left. The green halo marks the obstacle it is currently avoiding. The collision count is still zero.

5. RESULTS

We tested headless on nine fixed random seeds, 600 s (ten minutes) each. We count any contact at all between the robot and an obstacle or wall as a failure.

Metric	Result
Collision-free runtime	10 min, 0 collisions (9 seeds)
Obstacle rows cleared	all of them, endlessly
Difficulty stages reached	1 up to 17, then held
Average forward speed	about 0.6 m/s (slows to line up)
Active detours per 10 min	roughly 30 to 55
Physics rate	about 5,700 Hz

The robot uses the full width of the corridor as it follows a ball that is busy walking through the obstacles. On average the robot sits about 0.8 m to the side of the ball, and often more than 2 m, which is a simple check that it is really doing its own avoidance and not just trailing the ball. We also tried removing each of the three main ideas one at a time. Take away the sideways velocity,

the gate, or the guaranteed lane, and collisions come back. So all three are needed.

6. DISCUSSION AND LIMITATIONS

The walking policy is frozen and not fine-tuned for our task. It tracks sideways moves with about 0.25 m of error and trips now and then during the harder cross-corridor moves. We reduced these problems but did not remove them. The robot always got back up, and no trip turned into a collision in the runs we report. Vision is colour-based, so a change in lighting or ball colour would break it where a trained detector would not. Our test is a nine-seed sweep, not an exhaustive one. The obvious next steps are to fine-tune the policy for tighter sideways tracking, swap the

colour tracker for a learned detector, and plan one or two rows ahead so the approach to each gap is set up earlier.

7. CONCLUSION

A four-legged robot can cross a dense, endless obstacle corridor without a single collision for ten minutes using only a head camera and a walking policy it never trained. The two things that made it work were small: reuse the sideways velocity command the policy already had, and refuse to enter a gap before lining up with it. The wider lesson is that getting the most out of a general pre-trained policy can replace a lot of custom control and extra sensors.

REFERENCES

1. N. Rudin, D. Hoeller, P. Reist, M. Hutter. "Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning." *CoRL*, 2021. (source of our walking policy)
2. J. Hwangbo et al. "Learning agile and dynamic motor skills for legged robots." *Science Robotics*, 4(26), 2019.
3. J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, M. Hutter. "Learning quadrupedal locomotion over challenging terrain." *Science Robotics*, 5(47), 2020.
4. R. C. Coulter. "Implementation of the Pure Pursuit Path Tracking Algorithm." CMU-RI-TR-92-01, 1992.
5. G. R. Bradski. "Computer Vision Face Tracking for Use in a Perceptual User Interface (CamShift)." *Intel Technology Journal*, 1998.
6. E. Todorov, T. Erez, Y. Tassa. "MuJoCo: A physics engine for model-based control." *IROS*, 2012.